Micro-service prédiction
2026

1 Contexte

Dans le TP1, nous avons réalisé deux microservices Python :

- `clients` ;
- `commandes`.

Ces microservices communiquaient initialement avec une API REST. Dans le TP2, nous avons introduit Apache Kafka afin de permettre une communication asynchrone entre les microservices. Dans ce TP, nous allons ajouter un troisième microservice **Prediction**. Ce microservice utilisera un modèle de Machine Learning afin d'estimer la probabilité qu'un client passe une commande. L'objectif principal du TP est de comprendre comment intégrer un modèle de Machine Learning dans une architecture de microservices utilisant Kafka.

2 Objectifs

À la fin du TP, vous serez capables de :

- utiliser Kafka comme source de données pour un microservice ;
- créer un consommateur Kafka en Python ;
- utiliser un modèle de Machine Learning dans un microservice ;
- effectuer une prédiction à partir d'un événement Kafka ;
- publier le résultat de la prédiction dans un autre topic Kafka ;
- faire communiquer plusieurs microservices de manière asynchrone ;
- dockeriser le microservice de prédiction ;
- comprendre la différence entre apprentissage et prédiction.

3 Le problème de prédiction

- On souhaite prédire :

  Le client va-t-il passer une commande ?

- Le modèle reçoit plusieurs caractéristiques du client.
  Par exemple :

```
{
"client_id": 25,
"age": 32,
"anciennete": 3,
"nombre_commandes": 8,
"montant_total": 1250,
"montant_moyen": 156.25,
"jours_depuis_derniere_commande": 12
}
```

- Le modèle retourne par exemple :

```
{
"prediction": 1,
"probabilite": 0.84
}
```

Cela signifie que le modèle estime une probabilité de commande de 84

3.1 Préparer le modèle de Machine Learning

Le modèle peut être construit à partir des données historiques. Pour ce TP, utiliser un modèle de classification déjà entraîné, par exemple un arbre de décision ou une Random Forest. Le modèle doit être sauvegardé dans modele_commande.pkl. L'entraînement du modèle n'est pas réalisé à chaque événement Kafka. Le modèle est entraîné une fois puis sauvegardé. Le microservice `prediction` charge ensuite ce modèle.

3.2 Créer un microservice pour la Prediction

Créer le répertoire :

```
prediction/
|
+-- app.py
+-- modele_commande.pkl
+-- requirements.txt
+-- Dockerfile
```

Le fichier `requirements.txt` contient :

```
kafka-python
pandas
scikit-learn
joblib
```

3.3 Format d'un événement

Le microservice *commandes* doit produire un événement contenant les informations nécessaires à la prédiction. Par exemple :

```
{
"client_id": 25,
"age": 32,
"anciennete": 3,
```

```
"nombre_commandes": 8,
"montant_total": 1250,
"montant_moyen": 156.25,
"jours_depuis_derniere_commande": 12
}
```

3.4 Construire l'entrée du modèle

- À partir de l'événement Kafka, le micro-service prédiction construit le vecteur d'entrée qui pourrait ressembler à cela :

```
X = [[
data["age"],
data["anciennete"],
data["nombre_commandes"],
data["montant_total"],
data["montant_moyen"],
data["jours_depuis_derniere_commande"]
]]
```

- Pour utiliser, par exemple, un DataFrame :

```
import pandas as pd

X = pd.DataFrame([{
"age": data["age"],
"anciennete": data["anciennete"],
"nombre_commandes":
data["nombre_commandes"],
"montant_total":
data["montant_total"],
"montant_moyen":
data["montant_moyen"],
"jours_depuis_derniere_commande":
data["jours_depuis_derniere_commande"]
}])
```

3.5 Effectuer la prédiction

- Le micro-service effectue la prédiction :

```
prediction = model.predict(X)[0]
```

- Il calcule également la probabilité :

```
probabilite = model.predict_proba(X)[0][1]
```

- Il affiche :

```
print("Client :", data["client_id"])
print("Prediction :", prediction)
print("Probabilite :", probabilite)
```

3.6 Produire un événement de prédiction

- Après avoir effectué la prédiction, construire un nouvel événement :

```
resultat = {
"client_id": data["client_id"],
"prediction": int(prediction),
"probabilite_commande":
float(probabilite)
}
```

- Exemple :

```
{
"client_id": 25,
"prediction": 1,
"probabilite_commande": 0.84
}
```

- Cet évènement est destiné au micro-service *commandes*. Celui-ci doit afficher le résultat de la prédiction.

3.7 Questions avancées

— Dockeriser l'application.

— Prévoir un flux infini de demandes de prédictions et un nombre de micro-services *prédiction* égal à K, un paramètre de l'application.

— Tester la résilience à la panne en arrêtant un ou deux instance micro-services *prédiction*.

— Comment gérer la montée en charge des demandes de prédictions.